Full Stack Development with MERN Project

Documentation format

1. Introduction

- **Project Title:** BOOK A DOCTOR
- **Team Members:** Ishaq Ali

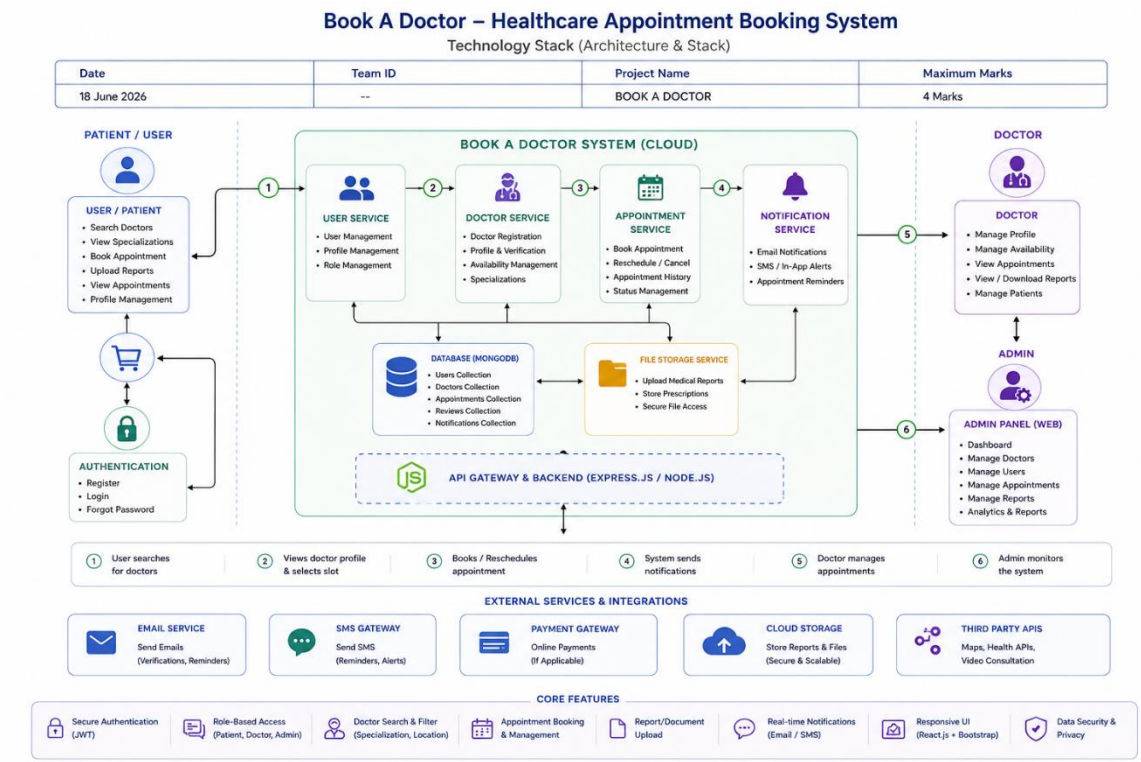
2. Project Overview

- **Purpose:**
  Book a Doctor is a healthcare appointment booking platform developed using the MERN stack (MongoDB, Express.js, React.js, and Node.js). The project aims to simplify the process of finding doctors and scheduling appointments through a secure and user-friendly web application. It enables patients to browse doctors, book appointments, upload medical reports, and manage their healthcare interactions digitally while allowing doctors and administrators to efficiently manage appointments and platform operations.
- **Features:**

- User Registration and Login
- Secure JWT Authentication
- Doctor Search and Filtering
- Doctor Profile Viewing
- Online Appointment Booking
- Appointment Management
- Medical Report Upload
- Patient Dashboard
- Doctor Dashboard
- Admin Dashboard
- Role-Based Access Control
- Responsive User Interface

3. Architecture

- **Frontend:** he frontend is built using React.js and Bootstrap, providing a responsive and dynamic user interface. React Router is used for navigation, while Axios handles API communication with the backend.
- **Backend:** The backend uses Node.js with Express.js to expose REST APIs for authentication, doctor management, appointments, and report uploads. Business logic is organized into controllers, routes, and middleware for maintainability.
- **Database:** MongoDB stores application data in collections such as Users, Doctors, Appointments, and Medical Reports. Mongoose is used for schema definition and database interactions.



4. Setup Instructions

- **Prerequisites:**
- □ Node.js (v16 or later)
- □ npm
- □ MongoDB or MongoDB Atlas
- □ Git
- **Installation:**

  - Clone the repository:

- git clone https://github.com/your-username/book-a-doctor.git

- Navigate to the project:

- cd book-a-doctor

- Install frontend dependencies:

- cd client

- npm install

- Install backend dependencies:

- cd ../server

- npm install

- Create a .env file:

- PORT=5000

- MONGO_URI=your_mongodb_connection_string

- JWT_SECRET=your_secret_key

5. Folder Structure

Client (React Frontend)

The **client** folder contains the React.js application responsible for the user interface and interactions. It is organized as follows:

```
client/
├── public/
├── src/
│   ├── assets/          # Images and static files
│   ├── components/      # Reusable UI components
│   ├── context/         # Authentication and global state
│   ├── pages/           # Application pages (Home, Login, Register,
Dashboard, etc.)
│   ├── services/        # API calls to the backend
│   ├── App.jsx          # Main application component
│   ├── main.jsx         # Entry point
│   └── index.css        # Global styles
├── package.json
└── vite.config.js
```

Server (Node.js Backend)

```
server/
├── config/              # Database configuration and environment setup
├── controllers/         # Handles request processing and business logic
├── middleware/          # Authentication and error-handling middleware
├── models/              # MongoDB/Mongoose schemas (User, Doctor,
Appointment)
├── routes/              # API route definitions
```

```
├── uploads/              # Uploaded files and images
├── utils/                # Helper and utility functions
├── .env                  # Environment variables
├── index.js              # Main server entry point
├── seed.js               # Database seed data
├── package.json
└── node_modules/
```

-

6. Running the Application

- Provide commands to start the frontend and backend servers locally.

Frontend:
cd client
npm start
Runs on:
http://localhost:3000
Backend:
cd server
npm start
Runs on:
http://localhost:5000

7. API Documentation

Method	Endpoint	Description
POST	/api/auth/register	Register a new user
POST	/api/auth/login	User login
GET	/api/doctors	Get all doctors
GET	/api/doctors/	Get doctor details
POST	/api/appointments	Book appointment
GET	/api/appointments/my	View user appointments
PUT	/api/appointments/	Update appointment

DELETE	/api/appointments/	Cancel appointment
POST	/api/reports/upload	Upload medical report
GET	/api/admin/dashboard	Get admin dashboard data

8. Authentication

Authentication is implemented using **JWT (JSON Web Tokens)**.

- Users log in with email and password.
- Passwords are securely hashed using **bcrypt**.
- JWT tokens are generated after successful login.
- Protected routes validate the JWT token before granting access.
- Role-based authorization ensures separate permissions for Patients, Doctors, and Admins.

9. User Interface

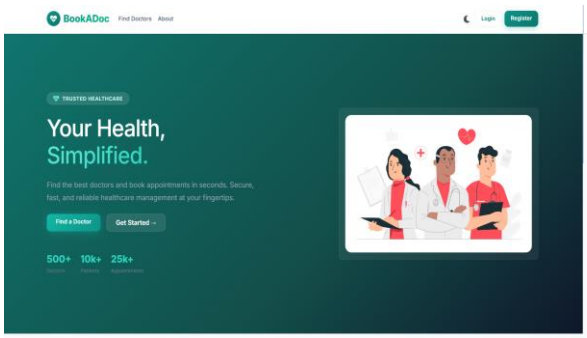
The application provides:

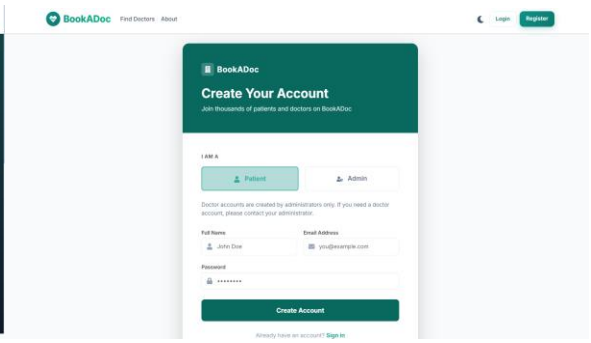
- Home Page
- Login & Registration
- Doctor Listing Page
- Doctor Profile Page
- Appointment Booking Page
- My Appointments Page
- Patient Dashboard
- Doctor Dashboard
- Admin Dashboard
- Profile Management Page

10. Testing

Testing Strategy

- **Functional Testing**
- **API Testing**
- **Authentication Testing**
- **CRUD Operation Testing**
- **UI Testing**
- **Responsive Design Testing**





11. Screenshots or Demo

Demo Description:

The Book a Doctor application allows patients to register, log in, search for doctors by specialization, view doctor details, book appointments, manage appointments, and update their profiles. Administrators can manage doctors, patients, and appointments through a secure dashboard.

12. Known Issues :

- Email or SMS notifications for appointments are not yet implemented.
- Online payment integration is currently unavailable.
- Video consultation functionality is not supported in the current version.
- Appointment reminders are not automatically sent.
- Search results may be limited if doctor profile information is incomplete.
- Mobile responsiveness can be further improved for smaller devices.
- ---

13. Future Enhancements

- Add video consultation and telemedicine support.
- Implement AI-based doctor recommendations based on symptoms.
- Enable appointment reminders through email, SMS, and push notifications.
- Add digital prescription management and medical record storage.

- Develop Android and iOS mobile applications.
- Include patient reviews and doctor ratings.
- Support multi-language interfaces for wider accessibility.
- Integrate hospital management systems and health insurance services.
- Enhance analytics and reporting features for administrators.